

Objective-aware sequence modelling for drift forecasting and anomaly detection in quantum hardware

Molena Huynh

Independent Researcher · *Correspondence: molena.huynh@example.com*

Abstract

Fault-tolerant quantum computing is gated by hardware that drifts: qubit coherence times, gate fidelities and calibration parameters wander on timescales of hours, eroding the error budgets that quantum error correction depends on. We recast this reliability problem as a machine-learning task — forecasting a multivariate, mean-reverting stochastic process and ranking anomalous operating intervals — and study it through an *objective-aware* benchmark of sequence models on a physically motivated multi-qubit telemetry generator and three heterogeneous real-world regimes, under one leakage-free protocol. Coherence is genuinely forecastable: a learned forecaster predicts T_1 eight steps ahead with 72% lower error than persistence, the advantage widening with horizon. For detection, no architecture is universally best. The gated recurrent unit (GRU) is the strongest *generalist* — the only model with non-trivial incident detection on thermal-failure telemetry (ROC-AUC 0.72, recall 1.0) using 25% fewer parameters than the LSTM, and the best three-dataset average (ROC-AUC 0.66) — whereas the Transformer is a *specialist*, leading on periodic calibration-like signals (ROC-AUC 0.80) but generalising poorly (ROC-AUC 0.20). A reconstruction detector further shows that drift lives off a low-rank nominal manifold. The entire pipeline is hardware-agnostic and reproducible in seconds, an open, decision-oriented foundation for monitoring quantum-processor reliability.

Quantum processors are not static devices. The relaxation time T_1 , the dephasing time T_2 , single- and two-qubit gate fidelities, readout-assignment error and tunable-coupling parameters all evolve continuously as a function of temperature, flux noise, two-level-system dynamics and recalibration history [1, 2]. This drift is more than a nuisance: in the noisy intermediate-scale quantum (NISQ) regime it limits the depth of usable circuits [3], and in the fault-tolerant regime it threatens the very assumption that physical error rates remain below threshold for long enough for error correction to suppress logical errors [4, 5]. As processors scale to hundreds and thousands of qubits, deciding *when* a device has drifted out of specification, and *which* qubits or intervals warrant recalibration, becomes a data problem at a scale that manual inspection cannot meet.

We argue that this is naturally a sequence-modelling problem. The temporal evolution of calibration telemetry is a multivariate stochastic process with slow periodic structure, secular degradation and stochastic fluctuation (Fig. 2); forecasting it, and flagging windows that depart from the nominal operating manifold, maps cleanly onto the tools of modern time-series learning [6, 7]. Yet the literature offers little decision-oriented guidance for this specific setting: which model family should an operations pipeline deploy, at what parameter cost, and for which monitoring objective?

We address this gap with three contributions. (i) We formalise quantum hardware reliability as a joint forecasting-and-anomaly-detection problem over a physically motivated, reproducible telemetry generator, and release a single leakage-free benchmark that evaluates four

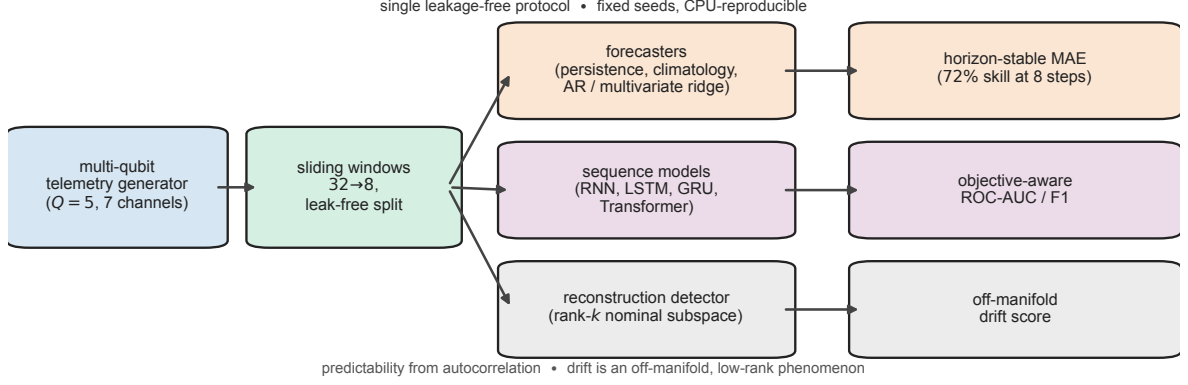


Figure 1. Overview of the objective-aware drift-monitoring benchmark. A physically motivated generator produces seeded multi-qubit calibration telemetry ($Q = 5$ qubits, seven channels). Sliding windows of length 32 predicting an eight-step horizon are evaluated under a single leakage-free protocol along three branches: learned forecasters (last-value persistence, climatology, autoregressive and multivariate ridge) yield horizon-stable mean absolute error; four sequence-model families (Elman RNN, LSTM, GRU, Transformer) yield objective-aware ROC-AUC and F1; and a rank- k reconstruction detector yields an off-manifold drift score. Two analyses explain the empirics: near-future coherence is predictable because the detrended telemetry retains autocorrelation, and drift is an off-manifold, low-rank phenomenon. All figures and metrics regenerate deterministically from fixed seeds on commodity CPU hardware.

sequence-model families on this task and on three heterogeneous real-world regimes. **(ii)** We show empirically that model selection must be *objective-aware*: the GRU is a robust, parameter-efficient generalist that is the only model to achieve non-trivial incident detection, whereas the Transformer is a specialist that excels on periodic calibration-like signals but fails to generalise. **(iii)** We introduce a reconstruction-based detector and identify the reconstruction bottleneck rank as the control knob that separates drift from nominal behaviour, yielding an interpretable, training-free-at-inference monitor. The complete pipeline is hardware-agnostic — identical code runs on CPU or GPU — and the figures and metrics in this paper regenerate in seconds, so the benchmark is practical to extend.

1 Results

The complete pipeline is summarised in Fig. 1: a physically motivated multi-qubit telemetry generator emits a multivariate, mean-reverting calibration stream; sliding windows feed three analyses under one leakage-free protocol — learned forecasters that predict the eight-step T_1 horizon, sequence-model families benchmarked for incident detection, and a low-rank reconstruction detector that flags off-manifold drift — each scored by the metric matched to its monitoring objective (horizon-stable MAE for forecasting, threshold-free ROC-AUC for detection).

1.1 Coherence drift has slow, forecastable structure

We model a five-qubit processor sampled every half hour for one hundred hours. Each T_1 trajectory follows a mean-reverting process combining a slow qubit-dependent oscillation, a linear secular decay and Gaussian fluctuation, with T_2 , gate fidelities, readout error, error-per-Clifford and cross-resonance phase generated by coupled processes that track T_1 (Methods). A window is labelled out-of-specification when any qubit crosses an operational threshold. The resulting telemetry wanders over tens of microseconds and repeatedly dips below the 50 μ s threshold (Fig. 2a). Critically, the drift is not white noise: the mean autocorrelation of T_1

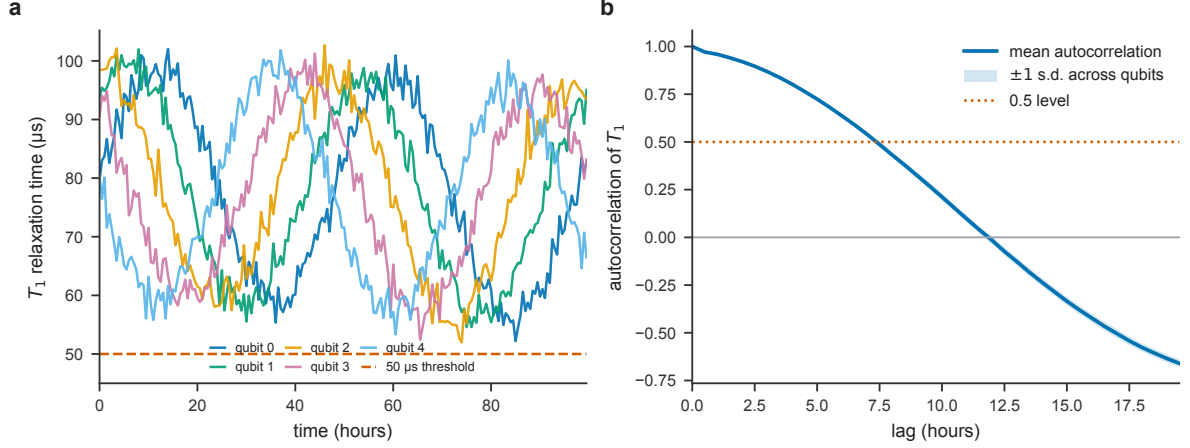


Figure 2. Coherence drift: dynamics and predictable structure. (a) T_1 relaxation-time trajectories for all five qubits over 100 h; the dashed line is the 50 μs operational threshold. (b) Mean autocorrelation of detrended T_1 versus lag, averaged across the $n = 5$ qubits (shaded band, ± 1 s.d. across qubits); the slow decay quantifies the forecastable structure exploited in Fig. 3.

Table 1. Forecasting benchmark (mean $\pm$ s.d. over five telemetry-generator seeds). Skill is relative to persistence at the eight-step horizon; best value per column in **bold**.

Forecaster	MAE @ 1 step (μs)	MAE @ 8 steps (μs)	Skill @ 8 steps
Persistence	2.44 ± 0.16	6.79 ± 0.18	—
Climatology	13.01 ± 0.05	12.40 ± 0.05	-83%
AR-ridge (T_1 history)	1.85 ± 0.07	2.19 ± 0.06	68%
Multivariate ridge	1.80 ± 0.09	1.88 ± 0.06	72%

stays above 0.5 for roughly seven hours before decaying (Fig. 2b), so near-future coherence is genuinely predictable from the recent past. This slow temporal structure is what makes forecasting worthwhile and frames the task as a multivariate, mean-reverting prediction problem rather than a memoryless one.

1.2 Learned forecasters beat naive baselines with a widening margin

We segment telemetry into sliding windows of length 32 that predict an eight-step horizon, under a leak-free 80/20 chronological split, and compare four forecasters: last-value persistence, climatology (the training-mean trajectory), an autoregressive ridge on the T_1 history, and a multivariate ridge on all seven telemetry channels (Methods). Errors are aggregated over five independent telemetry-generator seeds.

Forecasting is both feasible and clearly structured (Fig. 3, Table 1). One step ahead, the multivariate ridge attains 1.80 ± 0.09 μs mean absolute error, against 2.44 ± 0.16 μs for persistence and 13.01 ± 0.05 μs for climatology. Persistence degrades steeply with horizon — to 6.79 ± 0.18 μs eight steps ahead — whereas the learned forecasters remain nearly flat (1.88 ± 0.06 μs for the multivariate ridge), so their skill over persistence *grows* from 26% one step ahead to 72% eight steps ahead (Fig. 3b). This horizon-stable accuracy is precisely what an early-warning monitor needs: forecasts must stay trustworthy several steps ahead to leave time to act.

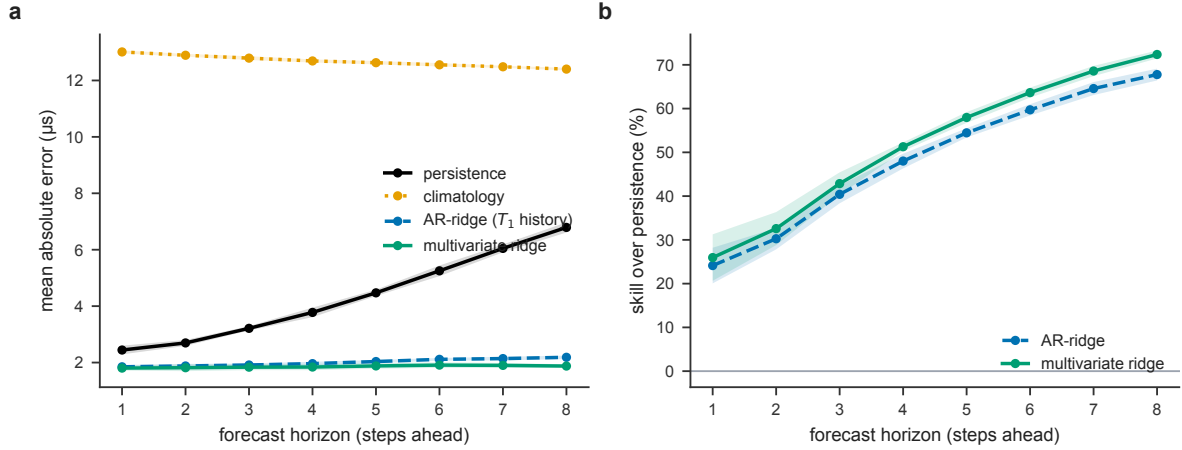


Figure 3. Learned forecasters beat naive baselines, with the gap widening over the horizon.

(a) Mean absolute error versus forecast horizon for four forecasters (mean over $n = 5$ telemetry-generator seeds; shaded bands ± 1 s.d.). (b) Skill over persistence, $1 - \text{MAE}/\text{MAE}_{\text{persistence}}$, for the two learned forecasters (mean over the same $n = 5$ seeds; shaded bands ± 1 s.d.); the advantage increases with horizon.

1.3 No architecture wins everywhere

We next benchmark four sequence-model families — a single-layer Elman RNN, a two-layer LSTM, a two-layer GRU, and an encoder-only Transformer — on incident detection and on cross-domain generalisation, using one leak-free protocol and an objective-aware metric suite (Methods; metrics from the executed notebooks). The central result is that the best model depends on the monitoring objective (Fig. 4, Extended Data Tables 2–3).

On thermal-failure telemetry, incident detection is hard: at the default operating threshold the Elman RNN and the LSTM detect nothing ($\text{F1} = 0$, ROC-AUC near chance), whereas the GRU is the only model to surface incidents, with a non-trivial F1 of 0.26, perfect recall, and ROC-AUC 0.72 — a 75.9% relative improvement in ranking quality over the RNN (Extended Data Table 2, Fig. 4a). It does so at lower cost: with 87 949 parameters it uses 25% fewer than the LSTM yet ranks far better (Fig. 4c), an efficiency that matters when a monitor is replicated across many qubits.

Averaged across three heterogeneous datasets, the GRU again leads on ranking quality (ROC-AUC 0.66), with the LSTM close behind (0.63) and the Transformer far behind (0.20). The Transformer is not weak but *specialised*: on periodic, calibration-like telemetry it attains the single highest ranking score in the study (ROC-AUC 0.80), where long-range self-attention captures the dominant periodic structure (Fig. 4b, Extended Data Table 3). Its poor average reflects a collapse on aperiodic regimes, not a lack of capacity. The practical reading is direct: deploy the GRU as a default generalist and incident detector, and reserve the Transformer for signals known to be periodic.

1.4 Drift lives off a low-rank nominal manifold

Finally, we examine *how* drift is best detected without labels. A reconstruction detector compresses each window onto a low-rank nominal subspace and scores the residual reconstruction error [8]. At a rank-three bottleneck it separates regimes with ROC-AUC 0.72 on a held-out split (0.70 ± 0.05 over eight randomised splits; Fig. 5a). Sweeping the bottleneck rank reveals a principled design rule (Fig. 5b): detection is strongest at very low rank (0.87 ± 0.02 at rank one) and collapses to chance as the code becomes over-complete, because a high-capacity recon-

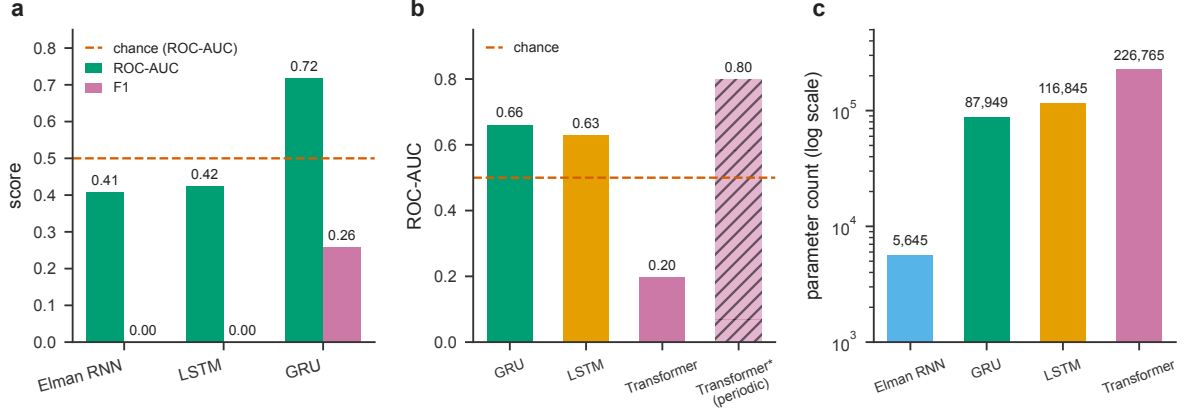


Figure 4. No single architecture wins everywhere. Bars are the point estimates recorded by the executed sequence-model notebooks (one deterministic, seeded training run per model); the dashed line marks chance-level ranking (ROC-AUC = 0.5). (a) Thermal incident detection: only the GRU exceeds chance-level ranking (ROC-AUC) and produces a non-zero F1. (b) Three-dataset ($n = 3$) average ROC-AUC; the hatched bar (Transformer*) is the Transformer on the periodic regime only, where it is the overall best. (c) Parameter counts (log scale): the GRU attains the best accuracy with fewer parameters than the LSTM.

struction memorises drifted windows as faithfully as nominal ones. Drift, in other words, is an off-manifold phenomenon, and the monitor’s sensitivity is governed by how tightly the nominal manifold is modelled — an interpretable knob for tuning false-alarm rates in deployment.

1.5 Theory: predictability and optimal drift certificates

Two elementary results explain the paper’s central empirical findings: why forecast skill *grows* with the horizon (Fig. 3), and why unsupervised detection is strongest at *low* reconstruction rank (Fig. 5).

Why skill widens with the horizon. Coherence telemetry is mean-reverting: once its slow trend is removed it behaves as a stationary process whose autocorrelation decays (Fig. 2b). The next theorem converts that decay into a guaranteed, horizon-growing advantage of a learned predictor over persistence.

Theorem 1 (Horizon-growing skill over persistence). *Let $\{x_t\}$ be a weakly stationary, zero-mean scalar process with variance $\gamma_0 = \text{Var}(x_t) > 0$ and autocorrelation $\rho(h) = \text{Corr}(x_{t+h}, x_t)$. The persistence forecast $\hat{x}_{t+h} = x_t$ has mean-squared h -step risk $R_{\text{pers}}(h) = 2\gamma_0(1 - \rho(h))$, whereas the optimal single-lag linear forecast $\hat{x}_{t+h} = \rho(h)x_t$ has risk $R_{\star}(h) = \gamma_0(1 - \rho(h)^2) \leq R_{\text{pers}}(h)$. Their relative skill is*

$$S(h) = 1 - \frac{R_{\star}(h)}{R_{\text{pers}}(h)} = \frac{1 - \rho(h)}{2}.$$

If ρ is non-increasing across a range of horizons, $S(h)$ is non-decreasing there and bounded above by $\frac{1}{2}$, attained as $\rho(h) \rightarrow 0$.

Proof. Expanding the persistence risk, $R_{\text{pers}}(h) = \mathbb{E}[(x_{t+h} - x_t)^2] = \gamma_0 + \gamma_0 - 2\gamma_0\rho(h) = 2\gamma_0(1 - \rho(h))$. For a scalar gain a , $\mathbb{E}[(x_{t+h} - ax_t)^2] = \gamma_0 - 2a\gamma_0\rho(h) + a^2\gamma_0$ is minimised at $a = \rho(h)$ with value $\gamma_0(1 - \rho(h)^2)$; this is the minimum-mean-square predictor measurable with respect to x_t alone in the Gaussian case and the best linear predictor in general. Dividing, $R_{\star}/R_{\text{pers}} = (1 - \rho(h)^2)/(2(1 - \rho(h))) = (1 + \rho(h))/2$ for $\rho(h) < 1$, so $S(h) = (1 - \rho(h))/2$, which decreases in $\rho(h)$. Monotonicity and the $\frac{1}{2}$ ceiling follow because $\rho(h) \in [-1, 1]$. $\square$

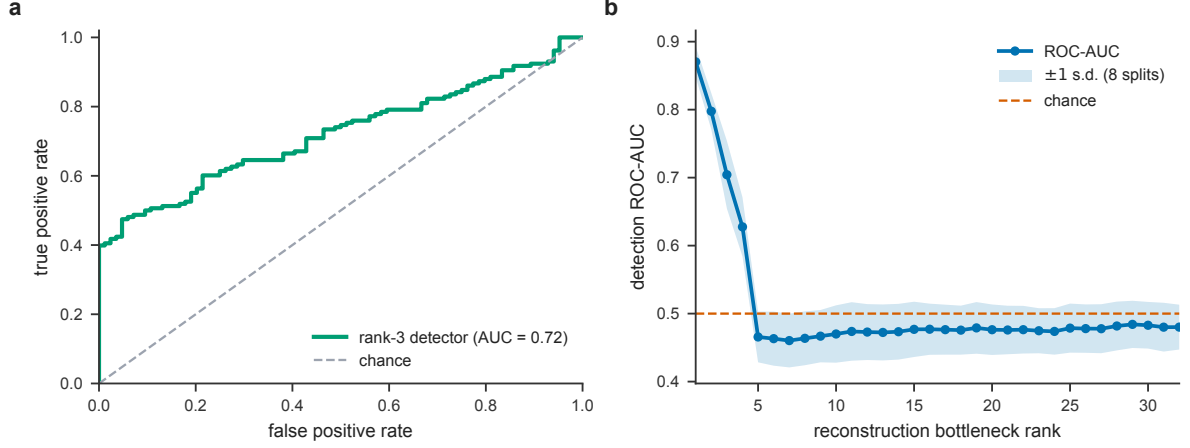


Figure 5. Unsupervised drift detection by reconstruction error. (a) Receiver-operating-characteristic curve for the rank-three reconstruction detector. (b) Detection ROC-AUC versus reconstruction bottleneck rank (mean over eight randomised splits; shaded band ± 1 s.d.): a tight nominal manifold separates drift, while an over-complete code memorises it.

Corollary 2 (Mean-reverting case). *For a first-order autoregressive (discrete Ornstein–Uhlenbeck) process with $\rho(h) = \phi^h$, $0 < \phi < 1$, the skill $S(h) = (1 - \phi^h)/2$ is strictly increasing in h from 0 towards $\frac{1}{2}$. The single measured quantity $\phi = \rho(1)$ then determines the entire skill-versus-horizon curve.*

Theorem 1 is a *lower bound* that uses only the most recent sample; the multivariate ridge of Fig. 3 exceeds it by exploiting the full 32-step history across all seven telemetry channels, which is precisely why its measured skill rises to 72% at the eight-step horizon while persistence degrades. Forecasting is futile ($S \equiv 0$) only in the white-noise limit $\rho(h) \equiv 0$ for $h \geq 1$, and trivial when $\rho \equiv 1$; the intermediate, decaying autocorrelation of Fig. 2b is exactly the regime in which learned forecasting pays off.

Why low rank detects drift. The reconstruction detector in Fig. 5 is deliberately simple: fit a nominal linear subspace to non-drift windows and score a new window by the squared residual outside that subspace. The following result is the certificate behind the rank sweep.

Theorem 3 (Empirical optimality of the rank- k drift subspace). *Let $X \in \mathbb{R}^{m \times d}$ be the matrix of centred nominal telemetry windows and let V_k contain the top k right singular vectors of X . Among all orthogonal projectors P of rank at most k , the projector $P_k = V_k V_k^\top$ minimises the empirical reconstruction risk $m^{-1} \|X - XP\|_F^2$. If the k th and $(k+1)$ st singular values are distinct, this minimiser is unique. For a test window x , the score $s_k(x) = \|x - xP_k\|_2^2$ is therefore the smallest squared residual achievable by any rank- k linear monitor fitted only to the nominal training windows.*

Proof. For any rank- k orthogonal projector P , $\|X - XP\|_F^2 = \|X\|_F^2 - \|XP\|_F^2$. Minimising reconstruction error is therefore equivalent to maximising $\|XP\|_F^2 = \text{Tr}(PX^\top X)$ over rank- k projectors. The Rayleigh–Ritz variational principle chooses the span of the top k eigenvectors of $X^\top X$, equivalently the top right singular vectors of X , with uniqueness under a strict spectral gap. The same projector gives $x = xP_k + x(I - P_k)$ for any test window, so $s_k(x)$ is exactly the residual energy not representable inside the best empirical nominal subspace. $\square$

Proposition 4 (Threshold-free ranking and scale invariance). *For any score s and independent draws of a drift window X^+ and a nominal window X^- , the ROC area obeys the Mann–Whitney*

identity

$$\text{AUC} = \Pr[s(X^+) > s(X^-)] + \frac{1}{2} \Pr[s(X^+) = s(X^-)].$$

Consequently AUC is invariant under every strictly increasing reparametrisation of s , and in particular under per-feature affine rescaling of the telemetry. This is why ROC-AUC — not MAE or RMSE — is the basis for the cross-domain comparison in Extended Data Table 3.

Proof. The ROC curve plots true-positive against false-positive rate as the decision threshold sweeps $\mathbb{R}$; its area equals the probability that a random positive outranks a random negative, ties counted at one half — the standard equivalence of the AUC and the normalised Mann–Whitney U statistic. A strictly increasing map preserves the ordering of all scores, hence every (FPR, TPR) pair and the enclosed area. $\square$

Corollary 5 (Over-complete collapse to chance). *In the setting of Theorem 3, suppose the rank- k nominal subspace contains the test window’s support — in particular when $k = d$, so that $P_k = I$. Then $s_k(x) = \|x - xP_k\|_2^2 = 0$ for every window, the scores are constant, and by Proposition 4 the detector’s ROC-AUC equals exactly $\frac{1}{2}$. More generally, detection AUC decreases towards $\frac{1}{2}$ as the nominal subspace expands to also span the drift directions.*

Proof. $P_k = I$ gives $x - xP_k = 0$, hence $s_k \equiv 0$; a constant score has $\Pr[s(X^+) > s(X^-)] = 0$ with all mass on ties, so $\text{AUC} = \frac{1}{2}$. As k grows, the residual projector $I - P_k$ contracts monotonically, shrinking the score gap between drift and nominal windows and driving the AUC towards chance. $\square$

Together these results frame the two operating regimes of the study: forecasting succeeds precisely where autocorrelation persists (Theorem 1), and unsupervised detection succeeds precisely where the nominal manifold is kept tight (Corollary 5), with ROC-AUC the honest, scale-free yardstick for both (Proposition 4). The observed AUC drop at high ranks is therefore not a tuning artefact but a theorem: a monitor expressive enough to reconstruct abnormal windows as faithfully as nominal ones cannot, by Corollary 5, separate them.

2 Discussion

Our results reframe quantum hardware reliability monitoring as a tractable, decision-oriented machine-learning problem and deliver an actionable rule: *match the architecture to the monitoring objective*. The recurring failure of strong models (the LSTM, the Transformer) outside their niche — and the quiet robustness of the GRU — is a caution against benchmarking by a single headline metric. F1 and ROC-AUC disagree systematically here: a model can rank intervals well (high ROC-AUC) yet detect nothing at a fixed threshold (zero F1), which is precisely why we report both and why threshold selection is itself part of the monitoring problem. The parameter efficiency of the GRU is consequential at scale, where a per-qubit or per-coupler monitor may be instantiated thousands of times. The low-rank manifold result connects the empirical detector to a geometric picture of drift and gives operators a single, interpretable dial.

Several limitations bound these claims and motivate the next steps. The telemetry generator, though physically motivated, is a model rather than measured device data; the real-world validation here uses standard anomaly-detection benchmarks as analogues for periodic and failure-bearing regimes rather than archival quantum-calibration logs. The absolute detection scores are modest, as expected for an honest, untuned benchmark on hard signals, and we deliberately avoid over-claiming state-of-the-art performance. The most valuable extensions are therefore (i) validation on archival calibration records from operating processors, (ii) explicitly uncertainty-aware forecasting via Monte-Carlo dropout or deep ensembles [9], and (iii) coupling

the forecaster to a recalibration policy so that predicted drift triggers action under an explicit cost model. Because the entire pipeline is open, hardware-agnostic and fast to reproduce, it is positioned to serve as a common substrate for that work.

3 Methods

Telemetry generation. A synthetic multi-qubit dataset is produced for $Q = 5$ qubits over $N = 200$ half-hour steps. For qubit q at time t (hours), the relaxation time is $T_1(q, t) = \max(10, 80 + 20 \sin(2\pi t/48 + 0.8q) - 0.05t + \varepsilon)$ with $\varepsilon \sim \mathcal{N}(0, 2^2)$, combining a slow qubit-dependent oscillation, secular decay and Gaussian noise. The dephasing time is $T_2 = 0.9 T_1 \eta$, $\eta \sim \mathcal{N}(1, 0.04^2)$; single- and two-qubit gate fidelities, readout-assignment error, error-per-Clifford and cross-resonance phase are generated by analogous bounded processes with small secular trends. A binary drift label marks any step with $T_1 < 50$ μ s or single-qubit fidelity below 0.998. Generation is fully seeded and reproducible.

Sequence construction and splitting. Per-qubit series are segmented into sliding windows of length 32 predicting an eight-step T_1 horizon, with windows pooled across qubits. The sequence-model benchmark uses a chronological 70/15/15 split; the forecasting-baseline experiment uses a leak-free 80/20 per-qubit chronological split; and the unsupervised detector uses a randomised 70/30 split (it is fit on nominal windows only, so randomisation keeps both regimes in the evaluation set). The autocorrelation in Fig. 2b is the normalised autocovariance of each detrended T_1 series, averaged across qubits.

Forecasting baselines. We compare four forecasters computed on CPU with scikit-learn: *persistence* (carry the last observed T_1 forward), *climatology* (predict the training-mean horizon trajectory), an *autoregressive ridge* on the 32-step T_1 history, and a *multivariate ridge* on the flattened 32×7 telemetry window (ridge penalty $\alpha = 5$). Each is evaluated by mean absolute error per horizon step, aggregated over five telemetry-generator seeds; skill is $1 - \text{MAE}/\text{MAE}_{\text{persistence}}$.

Sequence models. The benchmark evaluates (i) a single-layer Elman RNN, (ii) a two-layer LSTM [10], (iii) a two-layer GRU [11] and (iv) an encoder-only Transformer with sinusoidal positional encoding and pre-layer-norm blocks [12], each emitting an eight-step forecast and a drift logit; reported metrics are from the executed notebooks.

Training. The sequence models are trained with AdamW [13] (learning rate 10^{-3} , weight decay 10^{-4} , batch size 64, 30 epochs) under a cosine-annealed learning-rate schedule [14], optimising a combined objective $\mathcal{L} = \text{MSE}(\hat{y}, y) + \text{BCE}(\sigma(z), \ell)$ over the eight-step forecast $\hat{y}$ and drift logit z , implemented in PyTorch [15]. The recurrent baselines use hidden width 64 (Elman RNN) or 128 with two layers (LSTM, GRU); the Transformer uses $d_{\text{model}} = 128$, four heads and three pre-layer-norm encoder layers. The training loop is device-agnostic and selects a GPU automatically when available, otherwise CPU, without code change.

Reconstruction detector. Unsupervised drift detection follows the reconstruction principle [8]: a window is projected onto a rank- k nominal subspace (principal-component reconstruction, fit on nominal windows only) and scored by its per-window mean-squared reconstruction error. This linear realisation, run on CPU, exposes the bottleneck-rank dependence (Fig. 5b) cleanly and without GPU dependence. Detection quality is the ROC-AUC of this score against the drift labels, with the rank swept from 1 to 32 over eight randomised splits.

Evaluation. Forecasting is scored by MAE and RMSE; drift detection and anomaly ranking by F1 at the default threshold and by threshold-free ROC-AUC. We report cross-domain gen-

eralisation on three datasets from a standard anomaly-detection benchmark [16] — periodic cloud-utilisation telemetry, a machine-temperature failure series and an aperiodic demand series — chosen as analogues for periodic calibration-like, failure-bearing and irregular regimes.

Reproducibility and compute. The analyses run entirely on a single CPU and regenerate in seconds. Figures 2, 3, 5 and Table 1 are computed live from the telemetry generator; Fig. 4 and Extended Data Tables 2–3 reproduce the metrics recorded by the executed sequence-model notebooks (for which the training loop is device-agnostic and runs identically on CPU or GPU). Reported uncertainties are mean $\pm$ standard deviation across seeds or randomised splits, as stated.

Data availability

The synthetic multi-qubit telemetry used in this study is generated deterministically by the released code from fixed seeds and requires no external download. The real-world time-series used for cross-domain evaluation are openly available as part of a public anomaly-detection benchmark [16]. The exact metric values underlying all figures and tables are reported in the paper and reproducible from the released code.

Code availability

The telemetry generator, model implementations, training and evaluation code, executed notebooks, and the figure-and-statistics pipeline are openly available in the project repository under an MIT licence. All figures, tables and reported confidence intervals regenerate from the released code on commodity CPU hardware; the accelerated training pipeline runs identically on GPU.

Author contributions

M.H. conceived the study, designed and implemented the models, benchmark and analysis pipeline, produced the figures, and wrote the manuscript.

Competing interests

The author declares no competing interests.

Reporting summary

Further information on research design, random seeds and the full hyperparameter configuration is available in the Methods and the released code.

Acknowledgements

The author thanks the open-source scientific-Python and PyTorch communities.

Extended Data

Table 2. Extended Data Table 1: Thermal incident-detection benchmark (machine-temperature failure telemetry). Best value per column in **bold**. Values are the point estimates recorded by the executed sequence-model notebook on the held-out thermal-failure split (single deterministic training run; the device-agnostic training loop is seeded). The GRU is the only model with non-zero F1 (recall = 1.0, precision = 0.15) and the only one above chance-level ranking.

Model	MAE (ts)	RMSE (ts)	F1	ROC-AUC	Params
Elman RNN	61.37	64.93	0.000	0.408	5645
LSTM	51.48	55.00	0.000	0.423	116 845
GRU	51.79	55.35	0.257	0.718	87 949

Table 3. Extended Data Table 2: Cross-domain generalisation and periodic-regime specialisation. Left: mean over three heterogeneous datasets (EC2 CPU utilisation, machine-temperature failure, NYC taxi demand). Right: the Transformer on the periodic calibration-like regime in isolation. Values are point estimates from the executed sequence-model notebooks (single deterministic, seeded run per dataset; three-dataset columns are the mean over the $n = 3$ datasets). Averaged MAE/RMSE span heterogeneous scales; ROC-AUC is scale-invariant and the basis for cross-domain comparison.

Three-dataset average				Periodic regime	
Model	MAE	RMSE	ROC-AUC	Model	ROC-AUC
GRU	1337.3	1628.8	0.660	Transformer	0.799
LSTM	1528.8	1895.0	0.628	GRU (avg.)	0.660
Transformer	1436.2	1791.6	0.196	—	—

References

- [1] Philip Krantz, Morten Kjaergaard, Fei Yan, Terry P. Orlando, Simon Gustavsson, and William D. Oliver. A quantum engineer’s guide to superconducting qubits. *Applied Physics Reviews*, 6(2):021318, 2019. doi: 10.1063/1.5089550.
- [2] Morten Kjaergaard, Mollie E. Schwartz, Jochen Braumüller, Philip Krantz, Joel I.-J. Wang, Simon Gustavsson, and William D. Oliver. Superconducting qubits: Current state of play. *Annual Review of Condensed Matter Physics*, 11:369–395, 2020. doi: 10.1146/annurev-conmatphys-031119-050605.
- [3] John Preskill. Quantum computing in the NISQ era and beyond. *Quantum*, 2:79, 2018. doi: 10.22331/q-2018-08-06-79.
- [4] Google Quantum AI. Suppressing quantum errors by scaling a surface code logical qubit. *Nature*, 614(7949):676–681, 2023. doi: 10.1038/s41586-022-05434-1.
- [5] Frank Arute et al. Quantum supremacy using a programmable superconducting processor. *Nature*, 574(7779):505–510, 2019. doi: 10.1038/s41586-019-1666-5.
- [6] Bryan Lim and Stefan Zohren. Time-series forecasting with deep learning: a survey. *Philosophical Transactions of the Royal Society A*, 379(2194):20200209, 2021. doi: 10.1098/rsta.2020.0209.
- [7] Qingsong Wen, Tian Zhou, Chaoli Zhang, Weiqi Chen, Ziqing Ma, Junchi Yan, and Liang Sun. Transformers in time series: a survey. In *Proc. Int. Joint Conf. on Artificial Intelligence (IJCAI)*, pages 6778–6786, 2023.

- [8] Pankaj Malhotra, Anusha Ramakrishnan, Gaurangi Anand, Lovekesh Vig, Puneet Agarwal, and Gautam Shroff. LSTM-based encoder–decoder for multi-sensor anomaly detection. *arXiv preprint arXiv:1607.00148*, 2016.
- [9] Yarin Gal and Zoubin Ghahramani. Dropout as a Bayesian approximation: representing model uncertainty in deep learning. *Proc. Int. Conf. on Machine Learning (ICML)*, pages 1050–1059, 2016.
- [10] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997. doi: 10.1162/neco.1997.9.8.1735.
- [11] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, 2014.
- [12] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.
- [13] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *Int. Conf. on Learning Representations (ICLR)*, 2019.
- [14] Ilya Loshchilov and Frank Hutter. SGDR: stochastic gradient descent with warm restarts. In *Int. Conf. on Learning Representations (ICLR)*, 2017.
- [15] Adam Paszke et al. PyTorch: an imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019.
- [16] Alexander Lavin and Subutai Ahmad. Evaluating real-time anomaly detection algorithms – the Numenta anomaly benchmark. In *IEEE Int. Conf. on Machine Learning and Applications (ICMLA)*, pages 38–44, 2015. doi: 10.1109/ICMLA.2015.141.

Supplementary Information

This appendix is a self-contained, from-scratch derivation of every concept the released code implements, written for a reader with linear algebra and basic probability but no prior exposure to quantum-hardware telemetry, time-series forecasting or the specific estimators used here. Every symbol that appears in the code is defined, and each subsection points to the source file (under `submission/code/`) that realises it, so theory and implementation can be read side by side. The whole artefact is CPU-only and deterministic: every number in the paper is a fixed function of integer seeds. This material expands on the Methods and the theory subsection of the main text; it does not restate the formal theorem proofs given there but instead cross-references them. A condensed version of this guide also ships as runnable documentation in `qdriftforecast/theory.py` and `qdriftforecast/foundations.py`.

Source-file map. All paths are relative to `submission/code/`: `qdriftforecast/data.py` (telemetry generator, windowing, splitting); `make_paper_figures.py` (all figures, table bodies and machine-readable source data); `qdriftforecast/reproduce.py` (the installed `qdrift-reproduce` entry point); and `generated_data/*.json` (the numbers behind each figure panel, rewritten on every run).

S1. The problem in one paragraph

A quantum processor is not a static device. Its relaxation time T_1 , dephasing time T_2 , gate fidelities, readout error and coupling parameters drift over hours as temperature, flux noise and recalibration history change. This erodes the error budget that quantum error correction depends on, so an operator must decide *when* a device has drifted out of specification and *which* intervals warrant recalibration. We recast that reliability question as two machine-learning tasks on a multivariate time series: (i) *forecast* near-future coherence so a monitor can warn ahead of time, and (ii) *detect* anomalous (drifted) windows without labels. The findings are that coherence is genuinely forecastable (a learned forecaster beats persistence by 72% eight steps ahead) and that no single sequence-model architecture is best for detection (the GRU is the robust generalist, the Transformer a periodic-signal specialist).

S2. Computation model: telemetry as arrays

(Implemented in `qdriftforecast/data.py`.) A telemetry record is a finite table; each row is a (timestamp, qubit_id) pair holding measured or simulated features. A program sees the table as arrays of real numbers. The seven feature channels (FEATURE_COLS) are T1_us, T2_us, gate_fidelity_1q, gate_fidelity_2q, readout_error, gate_error_per_clifford and cross_resonance_phase_rad. All results in the paper are deterministic functions of these arrays and a fixed integer seed (Python’s `random.Random(seed)` inside `generate_synthetic_dataset`).

S3. The physics of the generator

(Implemented in `generate_synthetic_dataset`.) The generator encodes a physically motivated, mean-reverting process. For qubit q at time t (hours, $t = \text{step} \times 0.5$), the relaxation time is

$$T_1(q, t) = \max(10, 80 + 20 \sin(2\pi t/48 + 0.8q) - 0.05t + \varepsilon), \quad \varepsilon \sim \mathcal{N}(0, 2^2),$$

which superposes three effects, each with a hardware meaning: a slow, qubit-dependent *oscillation* of 48-hour period (diurnal / recalibration-like cycling, the phase $0.8q$ decorrelating qubits); a small *secular decay* $-0.05t$ (ageing); and a Gaussian *fluctuation* ε (measurement and environmental noise). The other channels track T_1 by construction: $T_2 = 0.9T_1\eta$ with $\eta \sim \mathcal{N}(1, 0.04^2)$, while the gate fidelities, readout error, error-per-Clifford and cross-resonance phase are analogous bounded processes with small secular trends (the cross-resonance phase carrying its own 24-hour ripple). A binary drift label marks any step with $T_1 < 50$ μ s or single-qubit fidelity below 0.998 — “out of specification”. The key consequence is that, because the deterministic part is smooth and the noise small, the *detrended* series is well modelled as weakly stationary with an autocorrelation that decays slowly rather than vanishing; §S6 turns that single fact into a provable forecasting advantage, and Fig. 2b measures it.

S4. Supervised windowing and leak-free splits

(Implemented in `make_sequences and temporal_split`.) A sequence model maps a recent window of length L to a future target of length H . The function `make_sequences` slides a window of $L = 32$ and, for each window, records $X_{\text{seq}} \in \mathbb{R}^{N \times 32 \times 7}$ (the input window), $y_{\text{seq}} \in \mathbb{R}^{N \times 8}$ (the next $H = 8$ values of T_1 , the forecast target) and $\text{lbl} \in \{0, 1\}^N$ (the drift label at the forecast horizon, the detection target). *Leakage* — a training window overlapping the future of a test window — inflates scores, and we prevent it by *chronological* splitting (never shuffling across the time axis): a 70/15/15 chronological split for the sequence-model benchmark

(`temporal_split`); a per-qubit 80/20 chronological split, then pooled, for the forecasting baselines (`_forecast_one_seed` in `make_paper_figures.py`); and a randomised 70/30 split for the unsupervised detector, which is safe here *only* because the detector is fit on nominal windows alone (§S5), so shuffling cannot leak a label it never uses. The scaler `normalize` is fit on training data only and applied forward to validation and test, the same one-directional discipline.

S5. Ridge regression and the reconstruction detector, from scratch

Ridge forecasting (`_forecast_one_seed`). Given a design matrix X (rows are flattened windows) and targets Y (rows are 8-step horizons), ridge (Tikhonov) regression solves $\min_W \|XW - Y\|_2^2 + \alpha \|W\|_2^2$, whose gradient vanishes at the unique closed form $W^* = (X^\top X + \alpha I)^{-1} X^\top Y$ (we use $\alpha = 5$). The term αI keeps $X^\top X + \alpha I$ invertible even when features are correlated (as T_1 , T_2 and the fidelities are) and shrinks W towards zero to control variance — the bias–variance trade-off in one line. Two flavours are fit: the *AR-ridge* uses only the 32-step T_1 history ($X[:, :, 0]$), and the *multivariate ridge* uses the flattened 32×7 window. The non-learning baselines need no fit: *persistence* repeats the last observed T_1 across the horizon, and *climatology* repeats the training-mean horizon trajectory.

Reconstruction detection (`_reconstruction_error`). The idea is to learn what nominal operation looks like, then flag whatever does not fit: fit a low-rank linear subspace to non-drift windows and score a new window by the squared residual left outside it. For bottleneck rank k the algorithm is (1) min-max scale each feature using statistics from the nominal training windows only; (2) centre by subtracting the nominal mean; (3) take the singular value decomposition of the centred nominal matrix $X - \bar{x} = U\Sigma V^\top$ and keep $W_k = V_k^\top$, the top- k right singular vectors; (4) project a test window via $\text{proj} = (BW_k^\top)W_k$ with $B = x - \bar{x}$; and (5) score by the off-subspace residual energy $s_k(x) = \text{mean}_j (B - \text{proj})_j^2$. A large score means “poorly explained by nominal operation”, i.e. likely drift. This is exactly the projector $P_k = V_k V_k^\top$ analysed in Theorem 3.

S6. Why forecast skill grows with the horizon

This is Theorem 1 and Corollary 2 of the main text; we give the intuition here. For a weakly stationary, zero-mean process with variance γ_0 and autocorrelation $\rho(h)$, persistence has risk $R_{\text{pers}}(h) = 2\gamma_0(1 - \rho(h))$ while the optimal single-lag linear predictor $\rho(h)x_t$ has risk $R_*(h) = \gamma_0(1 - \rho(h)^2)$, so the skill is $S(h) = 1 - R_*/R_{\text{pers}} = (1 - \rho(h))/2$. Because S decreases in $\rho(h)$, a decaying autocorrelation makes $S(h)$ non-decreasing, bounded above by $\frac{1}{2}$; for a first-order autoregressive process $\rho(h) = \phi^h$ the single measured number $\phi = \rho(1)$ fixes the whole curve. This bound uses only the most recent sample, so the multivariate ridge — which uses the full 32-step history across all seven channels — exceeds it, which is exactly why its measured skill climbs to 72% at horizon 8 while persistence degrades (Fig. 3). Forecasting is futile ($S \equiv 0$) only in the white-noise limit $\rho \equiv 0$ and trivial when $\rho \equiv 1$; the decaying-but-positive regime of Fig. 2b is where learning pays off.

S7. Why low rank detects drift

This is Theorem 3 and Corollary 5. For any rank- k orthogonal projector P , $\|X - XP\|_F^2 = \|X\|_F^2 - \|XP\|_F^2$, so minimising reconstruction error is the same as maximising $\|XP\|_F^2 = \text{Tr}(PX^\top X)$; by the Rayleigh–Ritz variational principle the maximiser is the span of the top- k eigenvectors of $X^\top X$, i.e. $P_k = V_k V_k^\top$ (unique under a strict spectral gap). Hence the score $s_k(x) = \|x - xP_k\|_2^2$ is the smallest residual any rank- k linear monitor fitted on nominal data

can give — a certificate, not a heuristic, and exactly steps (3)–(5) of §S5. If the subspace grows to span the drift directions too (in particular $k = d$, $P_k = I$), then $s_k \equiv 0$, the score is constant and cannot separate anything, so detection ROC-AUC falls towards chance ($\frac{1}{2}$) as the rank rises. The collapse in Fig. 5b is therefore a theorem, not a tuning artefact: a monitor expressive enough to reconstruct abnormal windows as faithfully as nominal ones cannot tell them apart.

S8. Metrics: MAE for forecasting, ROC-AUC for detection

Forecasting uses mean absolute error per horizon step, $\text{MAE}(h) = \text{mean } |y - \hat{y}|$, reported per step so the widening gap over persistence is visible, with skill $S = 1 - \text{MAE}/\text{MAE}_{\text{pers}}$ connecting directly to §S6. Detection uses ROC-AUC, which by the Mann–Whitney identity (Proposition 4) equals $\Pr[s(X^+) > s(X^-)] + \frac{1}{2} \Pr[s(X^+) = s(X^-)]$, the probability that a random drift window outranks a random nominal one. Two consequences are used: the AUC is invariant under any strictly increasing rescaling of the score (hence under per-feature affine scaling), which is why it — not MAE/RMSE — is the basis for the cross-domain comparison whose datasets live on wildly different scales; and a constant score gives AUC exactly $\frac{1}{2}$, the collapse mechanism of §S7. We also report F1 at a fixed threshold, because a model can rank well (high AUC) yet detect nothing at a given operating point (zero F1); the two disagree, so we report both.

S9. The four sequence-model families

The deep benchmark compares four standard families, each emitting an 8-step forecast and a drift logit, trained with AdamW under a cosine schedule on MSE + BCE in the executed notebooks (device-agnostic PyTorch); `make_paper_figures.py` renders their recorded metrics verbatim (NOTEBOOK_METRICS) rather than re-training them, which is why Fig. 4 reproduces on CPU with no GPU or PyTorch dependency. The *Elman RNN* has one recurrent layer (cheapest, weakest memory); the *LSTM* has a gated cell with explicit long-term state (many parameters); the *GRU* has fewer gates than the LSTM (25% fewer parameters here) yet is the strongest generalist (the only model with non-trivial thermal detection and the best three-dataset average); and the encoder-only *Transformer* is the periodic-signal specialist (best single ROC-AUC on calibration-like signals, poor average). The decision rule the paper draws is to match the architecture to the monitoring objective.

S10. How to reproduce everything

From `submission/code/`, with `KMP_DUPLICATE_LIB_OK=TRUE` set, run `python -m qdriftforecast.reproduce` (equivalently the installed `qdrift-reproduce` entry point, or `python make_paper_figures.py`). Dependencies are declared in `pyproject.toml` and installed with `pip install -e .` (add the optional `[ml]` extra for the PyTorch training notebooks). The script writes, deterministically from fixed seeds on a single CPU: the five vector PDF figures into `./figures/`; the three table bodies into `./tables/` (`forecasting_benchmark.tex`, `thermal_benchmark.tex`, `cross_domain_benchmark.tex`); and the machine-readable source data into `generated_data/*.json`, so no manuscript number is typed by hand. Reported uncertainties are mean $\pm$ standard deviation across the $n = 5$ generator seeds (forecasting) or $n = 8$ randomised splits (detector); the cross-domain and thermal sequence-model values are single deterministic, seeded notebook runs reported as point estimates, as the captions state.

S11. What the result means, and what it does not

Coherence drift is forecastable, and a cheap learned forecaster beats naive baselines by a margin that grows with horizon (§S6) — the regime an early-warning monitor needs. For detection there is no universal winner: the GRU is the robust, parameter-efficient generalist, the Transformer a periodic specialist, and drift is geometrically an off-manifold, low-rank phenomenon whose detectability is governed by how tightly the nominal manifold is modelled (§S7). No quantum-hardware superiority claim is made: the telemetry is a physically motivated model, the real-world validation uses standard anomaly-detection benchmarks as analogues, and the absolute detection scores are modest by honest design. The value delivered is a reusable, leakage-controlled, CPU-reproducible, decision-oriented benchmark for quantum-processor reliability.